it's been almost ayear from my graduation (technicien specialise 2 year in web dev) i graduated 06/2024 and now we are in 05/2025, and i did a big mistake that is i didn't do nothing the whole this period no coding no certificates nothing i was completly lost and now all of my calssmates i study with are doing good some got a job some intership some freelancing some continue his study (licence pro +1 year) and me still at the same point / level as i was in 06/2024 when i graduate even my coding skills are getting

Thought for 15 seconds

Okay, the user is feeling really stuck. They graduated almost a year ago with a technical degree in web development but haven't done anything since—no coding, certificates, or further education. Now they're seeing their classmates move forward with jobs, internships, freelancing, or more education, while they feel like they're at the same level or even worse. They just started relearning CSS, JS, and React. They need advice on how to get back on track.

First, I should acknowledge their feelings. It's important they feel heard. Then, maybe break down the problem. They took a gap year unintentionally, but it's not too late to start again. They need actionable steps. Let's see... Maybe start with self-compassion, then practical steps like rebuilding skills, creating projects, networking, job search strategies, learning from peers, staying consistent, and considering further education if needed.

I should emphasize that it's okay to take a break and that progress isn't linear. Comparing to others can be demotivating, so focusing on their own journey is key. Suggest setting small goals, building a portfolio, contributing to open source, freelancing platforms. Maybe mention online courses or certifications they can pursue now. Also, using LinkedIn and networking to find opportunities. Maybe mock interviews or practice for job applications. Encouraging consistency, like coding daily, even a little. And perhaps a timeline to measure

> progress in 3-6 months. Ending on a positive note that it's not too late and many have bounced back from similar situations.

It’s completely normal to feel stuck or lost after graduation, especially when comparing yourself to others. But here’s the good news: **it’s not too late to turn things around**, and your awareness of the situation is the first step toward progress. Here’s how to reframe your mindset and take action:

1. Stop Comparing Yourself to Others

- Your classmates’ journeys are *theirs*—not yours. Everyone progresses at their own pace. Some people land jobs quickly, others pivot later, and many take breaks (intentionally or not). Focus on **your path**, not theirs.
- Social media/LinkedIn only show highlights, not the struggles behind the scenes. Many of your peers might also feel insecure or overwhelmed.

2. Forgive Yourself & Reset

- Shame and guilt are paralyzing. Accept that the past year happened, but recognize that **today is a new starting point**. You’re already taking action by relearning CSS, JS, and React—that’s progress!
- Think of this as a "gap year" to reflect. Many people take breaks and still succeed—your career is a marathon, not a sprint.

3. Rebuild Your Skills Strategically

- **Start small** to avoid burnout. For example:
  - Daily coding: Spend 1–2 hours on platforms like freeCodeCamp, Codecademy, or Frontend Mentor to rebuild muscle memory.
  - Focus on projects, not tutorials: Build a portfolio (even if it's simple). Redo your graduation project or clone real-world apps (e.g., a Twitter clone with React).
  - Learn in public: Share your progress on LinkedIn or Twitter—it holds you accountable and attracts opportunities.
- **Certifications**: Pursue free/affordable ones (e.g., Google UX Design, Meta Front-End Developer) to fill gaps and boost your resume.

4. Create Opportunities

- **Freelance**: Start small on platforms like Upwork or Fiverr. Even simple tasks (fixing CSS bugs, building static sites) can rebuild confidence and skills.
- **Open Source**: Contribute to GitHub projects. It's a great way to learn collaboration and modern workflows.
- **Internships/Volunteer Work**: Reach out to local businesses or nonprofits to build real-world experience. Many will accept free help for small projects.

5. Network & Learn from Others

- Reconnect with classmates or mentors. Ask for advice, not comparisons. Many will be happy to help.
- Join coding communities (Discord, Reddit, local meetups) to stay motivated and learn about job openings.
- Follow developers on YouTube or Twitter for inspiration (e.g., Traversy Media, Kevin Powell, Wes Bos).

6. Job Search Tactics

- Tailor your resume/LinkedIn to highlight your technical diploma and any projects (even academic ones).
- Apply for entry-level roles like **Junior Front-End Developer**, **IT Support**, or **Web Maintenance**—these often value foundational skills over experience.
- Prepare for interviews by practicing algorithms (LeetCode, HackerRank) and behavioral questions (STAR method).

7. Stay Consistent & Track Progress

- Set **SMART goals** (e.g., “Build a React app in 2 weeks” or “Apply to 5 jobs/week”).
- Use tools like Trello or Notion to track daily tasks and celebrate small wins.
- Avoid “all or nothing” thinking—coding 30 minutes a day is better than zero.

8. Consider Further Education (If It Aligns with Your Goals)

- If a “Licence Pro” or bootcamp feels right, research programs that offer job placement support. But only pursue this if you’re committed—**self-learning can be equally powerful**.

Final Thoughts:

- **Your skills aren’t gone**—they’re just rusty. With consistent practice, they’ll come back faster than you think.
- The tech industry values problem-solving and adaptability over perfection. Employers care more about what you can build than gaps in your resume.
- Progress won’t happen overnight, but in 3–6 months of focused effort, you’ll see dramatic improvements.

You’ve already taken the hardest step: deciding to start again. Now, just keep moving forward—one line of code at a time. 💪

> what if i did not find any job or intership what should i do also how can i avoid that comparison it really kills me and demotivate me also tell me everything i must do to not ruin my tech carrier ?

It’s understandable to feel anxious about the future, especially when facing uncertainty in your career. But let’s break this down into **actionable steps** to ensure you don’t ruin your tech career—and, more importantly, to **rebuild it stronger than before**.

1. What If You Don’t Find a Job or Internship?

If traditional paths (jobs/internships) don’t work out immediately, **create your own opportunities**:

A. Freelancing & Gig Work

- Start on **Upwork, Fiverr, or even Facebook groups** offering:
  - Simple website fixes (HTML/CSS tweaks).
  - WordPress site setups.
  - Basic JavaScript automation scripts.
- **How to start?**
  - Offer free/cheap work for testimonials.
  - Build a portfolio (even if it’s just 2-3 projects).

B. Open Source & Volunteer Work

- Contribute to **GitHub** (start with "good first issue" tags).
- Offer free web dev help to **local NGOs, mosques, or small businesses** (real projects = real experience).

C. Build & Sell Something

- Create a **small SaaS tool** (e.g., a resume builder, habit tracker).

- Sell templates (Themes, UI kits) on **Gumroad or Etsy**.

D. Remote Internships & Apprenticeships

- Apply for remote programs like:
  - Outreachy (paid remote internships).
  - Google Summer of Code.
  - Microverse (coding bootcamp with deferred payment).

2. How to Stop Comparison (It's Killing Your Motivation)

A. Accept That Everyone's Path Is Different

- Some people get jobs fast because of **luck, connections, or grinding non-stop**.
- Others take years but end up in better positions.
- **Your journey is yours alone**—comparison only steals your joy.

B. Limit Social Media/LinkedIn

- Unfollow people who trigger insecurity.
- Remind yourself: **People only post wins, not struggles.**

C. Focus on "1% Better Every Day"

- Instead of thinking: *"They're ahead of me,"* ask:
  - *"What's one thing I can learn today?"*
  - *"What project can I finish this week?"*

D. Track Your Own Progress

- Keep a **coding journal** (what you learned, built, applied to).
- Celebrate small wins (e.g., *"Today I fixed a bug in my React app!"*).

3. Everything You Must Do to Not Ruin Your Tech Career

A. Never Stop Coding (Even If Unemployed)

- **Daily coding habit** (30 mins > 0 mins).
- Rebuild skills with:
  - Frontend Mentor (real-world projects).
  - The Odin Project (full-stack path).

B. Build a Strong Portfolio

- 3-4 solid projects (e.g., an **e-commerce site, a CRUD app, a clone of Twitter/Spotify**).
- Deploy them on **Vercel/Netlify** + GitHub.

C. Network the Right Way

- Reach out to **senior devs on LinkedIn** (ask for advice, not jobs).
- Join **Discord groups, local meetups, hackathons**.

D. Learn In-Demand Skills

- **Must-know in 2025**:
  - React/Next.js
  - TypeScript
  - Basic backend (Node.js + Express)
  - SQL (PostgreSQL) or Firebase
  - Git + GitHub

E. Apply Strategically

- **Customize your resume** for each job (use keywords from the job post).
- Apply to **at least 5 jobs/week** (even if you're underqualified).
- Prepare for interviews:
  - **Behavioral**: "Tell me about a time you solved a problem..."
  - **Technical**: Practice on LeetCode (Easy problems first).

4. Mindset Shift: This Is Just a Delay, Not a Failure

- Many successful devs had gaps, career changes, or slow starts.
- **The only way you ruin your career is by quitting.**
- **Right now, you're in the "struggle phase" that every successful person goes through.**

Final Advice:

1. **Code every day** (even if just 30 minutes).
2. **Build projects** (they replace lack of experience).
3. **Network** (most jobs come from connections).
4. **Apply even if you're scared** (rejection is part of the process).
5. **Ignore the noise**—your journey is yours alone.

You're not behind. You're just getting started. 🚀 Keep going.

> okay now i need advice so i can move fast in coding because i see my self with a low progress, slow learning, slow moving to next steps, slow practicing i take a lot of time

I hear your frustration—you want to **speed up your progress** in coding, but it feels like you're moving too slowly. The good news? **You can optimize your learning process** to move faster while still retaining knowledge. Here's a **step-by-step action plan** to accelerate your coding skills effectively.

1. Fix Your Learning Strategy (Stop Wasting Time)

🚫 Common Mistakes That Slow You Down:

- **Tutorial Hell** – Watching endless videos without building anything.
- **Jumping Between Topics** – Learning React, then Python, then random APIs without focus.
- **No Clear Goals** – "I'll learn JavaScript" is too vague.

✅ How to Learn Faster:

✓ **Focus on ONE Stack** (e.g., **MERN: MongoDB, Express, React, Node.js**)
✓ **Learn by Building** (Stop passive learning—start coding immediately)
✓ **Use the 80/20 Rule** – Learn the **20% of concepts that give 80% of results** (e.g., in React: components, state, hooks, API calls).
✓ **Set Deadlines** – *"I'll build a full CRUD app in 1 week."*

2. Speed Up Your Practice (Code Every Day Like a Pro)

🔥 Daily Routine for Fast Progress:

Time	Activity	Goal
30 min	**LeetCode / Codewars** (Easy problems)	Improve logic & speed

Time	Activity	Goal
1-2 hrs	**Build a Mini Project** (e.g., Todo App, Weather App)	Apply concepts
30 min	**Review & Refactor Code** (Find better ways to write what you did)	Learn from mistakes
20 min	**Watch a Short Tutorial (ONLY if stuck)**	Fill knowledge gaps

⚡ How to Practice Smarter:

- **Code Along, Then Build Alone** (Don't just copy—understand, then recreate from scratch).
- **Break Projects into Tiny Steps** (Example for a Todo App):
  1. Set up React.
  2. Add input field + button.
  3. Save tasks in state.
  4. Display list.
  5. Add delete function.
- **Use ChatGPT for Debugging** (Ask: *"Why is my React state not updating?"*)

3. Build Real Projects Fast (Portfolio Boost)

🚀 Project Ideas to Speed Up Learning:

Project	Skills Practiced	Time Goal
Todo App	React State, CRUD	2 days
Weather App	API Fetching, Async JS	3 days
E-commerce (Fake Store)	React, Cart Logic	1 week
Twitter Clone	Firebase Auth, Posts	1.5 weeks

📌 Rules for Fast Project Building:

✓ **No Perfectionism** – Ugly but functional > Unfinished "perfect" code.
✓ **Deploy Every Project** (Use **Vercel/Netlify** + GitHub).
✓ **Add to LinkedIn/Portfolio Immediately** (Even if simple).

4. Learn Like a Hacker (Retain Knowledge Faster)

🧠 Active Learning Techniques:

- **Feynman Technique** – Explain a concept in simple terms (like teaching a kid).
- **Spaced Repetition** – Revisit old code/projects every few days.
- **Anki Flashcards** (For key concepts like *"What is useEffect?"*).

📚 Best Free Resources for Fast Learning:

- **JavaScript/React:** The Odin Project (Project-based)
- **UI/UX:** Frontend Mentor (Real-world designs)
- **Algorithms:** NeetCode.io (LeetCode roadmap)

5. Avoid Burnout (Stay Consistent)

- **Pomodoro Technique** (25 min coding, 5 min break).
- **No Zero Days** – Even 20 mins of coding counts.
- **Reward Progress** (Finished a project? Treat yourself!)

Final Checklist to Move Faster:

1. **Pick ONE stack** (e.g., MERN).
2. **Code daily** (LeetCode + Projects).
3. **Build & deploy 1 project per week.**
4. **Review & refactor code** (Learn from mistakes).
5. **Network & apply for jobs** (Even if you feel "not ready").

💡 Remember:

- **Speed comes with repetition**—the more you code, the faster you get.

- **Done is better than perfect**—ship projects, then improve.
- **Comparison kills progress**—focus on your own journey.

You got this! 🚀 Start today, and in **3 months**, you'll look back amazed at your progress.

> what about my classmates we will see my beginner level projects on github , should i just don't care ! what's your thoughts

Should You Care About Your Classmates Seeing Your Beginner Projects?

Short answer: **No, don't worry about it.** But let's break this down so you feel confident in your approach.

1. Why You Should NOT Be Embarrassed by Beginner Projects

✅ **Everyone Starts Somewhere** – Even senior devs had cringe-worthy early projects.
✅ **Progress > Perfection** – Your GitHub shows **growth**, not just skill level.
✅ **Recruiters Prefer Active Learners** – A GitHub with **consistent commits** (even small ones) looks better than an empty one.
✅ **Your Classmates Aren't Judging You** – Most are too busy with their own journeys.

Example:

- **June 2024:** Basic HTML/CSS project
- **July 2024:** Simple JS calculator
- **August 2024:** React Todo App
- **September 2024:** Full-stack CRUD app
  → This **progression** is what matters, not where you started.

2. How to Handle the Mental Block

A. Shift Your Mindset

- **Instead of:** *"They'll think I'm a noob."*
- **Think:** *"I'm documenting my coding journey. This is how improvement works."*

B. Make Your GitHub Work for You

- **Pin your best 4-6 projects** (even if simple).
- **Write clean READMEs** (Explain what you learned in each project).
- **Delete/archive old experiments** if they no longer represent your skills.

C. Turn "Weakness" Into Strength

If someone asks:

- *"Why do you have beginner projects?"*
- **Your reply:**

> *"I believe in learning in public. These were my first steps—now I'm building [current project]. Want to see?"*

3. What Your Classmates (and Employers) Actually Think

Scenario	What They Really See
Empty GitHub	*"Did they stop coding after graduation?"*
Beginner Projects + Recent Progress	*"They're actively learning and improving."*
Only Fancy Projects (But No Recent Activity)	*"Did they buy this portfolio?"*

Truth: A **messy-but-active GitHub** is **100x better** than a silent one.

4. What You Should Do Now

1. **Keep pushing code** (even if it's small fixes).
2. **Gradually replace old projects** with newer, better ones.
3. **Engage with the dev community** (Comment on others' repos, share your work).
4. **Own your journey** – No successful dev regrets their early projects. They **used them as stepping stones.**

Final Thought:

Your GitHub is **your lab notebook**, not a trophy case.

Elon Musk's first website looked like this:

https://i.imgur.com/5v8Zm9O.png

→ Did he care? No. He kept building.

Keep coding. Keep shipping. The only bad GitHub is an empty one. 🚀

> if i make repository private but i always pushing to github does it count on my github profile that i've been active (i am talking about the green ones)

Do Private Repositories Count Toward GitHub Activity (Green Squares)?

Short answer: Yes! Private commits *do* show up in your GitHub contribution graph (the green squares) **as long as:**

- You're the **owner** of the private repo (or a collaborator).
- The repo is **not a fork**.

What Counts (and What Doesn't)

Scenario	Shows in Green Squares?
Commits in a private repo (you own)	✅ **YES**

Scenario	Shows in Green Squares?
Commits in a private fork	❌ No
Commits in someone else's private repo	❌ No (unless you're a collaborator)
Public repo commits	✅ **YES**

How to Verify Your Private Contributions

1. Go to your **GitHub profile page** (`github.com/yourusername`).
2. Scroll to the **contribution graph** (green squares).
3. Click on a **recent day** – you should see private commits marked with a **lock icon** 🔒.

Should You Keep Repos Private or Public?

Private Repos Are Good For:

- **Personal projects you're not ready to share**
- **Experimenting without judgment**
- **Job-specific code (NDA/sensitive work)**

Public Repos Are Good For:

- **Showcasing skills to employers**

- **Networking (other devs can see your work)**
- **Contributing to open source**

👉 **Best Strategy:**

- Keep **learning/practice code** private if you prefer.
- Gradually make **3-5 strong projects public** for your portfolio.

Final Tip:

GitHub activity **isn't just about green squares**—it's about **consistent progress**.
Even if you keep repos private:

- **Push code daily** (builds discipline).
- **Later, make select projects public** when polished.

Your future self will thank you for not overthinking it and just coding! 💻🚀